

Ответы на обычные вопросы по БД

Кратко, по сути, с сохранением ключевых деталей.

1. Оператор SELECT. Ограничение выборки строк и столбцов. Удаление строк-дубликатов. Сортировка.

SELECT предназначен для выборки данных из таблиц. Ограничение столбцов выполняется перечислением нужных полей после SELECT или символом * для всех столбцов. Ограничение строк выполняется WHERE с операторами =, <>, >, <, BETWEEN, IN, LIKE, IS NULL и логическими AND, OR, NOT. DISTINCT удаляет повторяющиеся строки результата. ORDER BY задает сортировку, ASC используется по умолчанию, DESC - по убыванию; сортировать можно по нескольким столбцам.

2. Оператор SELECT. Объединение таблиц с помощью WHERE и JOIN. Декартово произведение. Внешнее соединение. Соединение таблицы с собой.

Соединение таблиц нужно для получения связанных данных из нескольких таблиц. Старый способ - перечислить таблицы в FROM и задать условие соединения в WHERE; современный - JOIN ... ON. Если условие соединения отсутствует, получается декартово произведение: каждая строка первой таблицы соединяется с каждой строкой второй. Внешнее соединение LEFT/RIGHT/FULL сохраняет строки без пары из одной или обеих таблиц. Самосоединение выполняется через два псевдонима одной таблицы, например для связи сотрудника и его руководителя.

3. Использование подзапросов. Однострочные и многострочные подзапросы.

Подзапрос - SELECT, вложенный в другой SQL-оператор, обычно в WHERE, HAVING или FROM. Он заключается в скобки, выполняется для получения значения или набора значений, которые использует внешний запрос. Однострочный подзапрос возвращает одну строку и применяется с =, >, <, >=, <=, <>. Многострочный возвращает несколько строк и требует IN, ANY или ALL; использование = с таким подзапросом приводит к ошибке.

4. Функции SQL. Однострочные и многострочные функции. Функция TO_CHAR с датами.

Функции SQL принимают аргументы и возвращают результат. Однострочные функции работают с каждой строкой отдельно: символьные LOWER, UPPER, SUBSTR, LENGTH; числовые ROUND, TRUNC; функции дат SYSDATE, ADD_MONTHS, MONTHS_BETWEEN; преобразования TO_CHAR, TO_DATE, TO_NUMBER; общие NVL, CASE, DECODE. Многострочные функции обрабатывают группу строк и возвращают одно значение: COUNT, SUM, AVG, MIN, MAX. TO_CHAR(date, 'format') преобразует дату в строку, например DD.MM.YYYY или DD-MONTH-YYYY HH24:MI:SS.

5. Группировка записей в SQL. Групповые функции. Ограничение вывода сгруппированных данных.

GROUP BY разбивает строки на группы по значениям указанных столбцов. К каждой группе можно применять COUNT, SUM, AVG, MIN, MAX и другие групповые функции. В SELECT при группировке допускаются только столбцы из GROUP BY и групповые выражения. WHERE

фильтрует строки до группировки, HAVING фильтрует уже сформированные группы. ORDER BY можно использовать после HAVING для упорядочивания результата.

6. Команды языка манипулирования данными (DML).

DML - язык манипулирования данными. Основные команды: SELECT выбирает данные, INSERT добавляет строки, UPDATE изменяет существующие строки, DELETE удаляет строки, MERGE совмещает вставку и обновление по условию. DML выполняется в рамках транзакций: изменения становятся окончательными после COMMIT и отменяются ROLLBACK. В отличие от DDL, DML обычно не выполняет неявную фиксацию автоматически.

7. Команды языка определения данных (DDL). Использование команд ALTER, TRUNCATE, DROP для таблиц.

DDL управляет структурой объектов БД: CREATE создает объект, ALTER изменяет, DROP удаляет, TRUNCATE быстро очищает таблицу. ALTER TABLE добавляет, изменяет и удаляет столбцы или ограничения. TRUNCATE TABLE удаляет все строки без построчного журналирования и обычно быстрее DELETE, но откат ограничен. DROP TABLE удаляет таблицу как объект вместе с ее данными и зависимыми объектами по правилам СУБД.

8. Создание таблицы с определением всех столбцов. Создание таблицы на основании данных из другой таблицы. Изменение структуры таблицы.

Таблица создается CREATE TABLE с указанием имен столбцов, типов данных и ограничений. Можно задать NOT NULL, PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK, табличное пространство и параметры хранения. Таблица по данным другой таблицы создается CREATE TABLE new_table AS SELECT ...; при этом копируются данные и структура выбранных столбцов, но не все ограничения. Структура меняется ALTER TABLE ADD, MODIFY, DROP COLUMN, RENAME COLUMN и ADD/DROP CONSTRAINT.

9. Объекты схемы SQL. Представления. Простые и сложные представления.

Схема содержит объекты пользователя: таблицы, представления, индексы, последовательности, процедуры, функции, пакеты, триггеры, синонимы. Представление - сохраненный запрос, который выглядит как таблица, но обычно не хранит собственные данные. Простое представление строится по одной таблице, без группировки и сложных выражений, часто допускает DML. Сложное представление содержит соединения, групповые функции, GROUP BY, DISTINCT или вычисляемые выражения; изменение данных через него ограничено.

10. Объекты схемы SQL. Последовательности.

Последовательность - объект схемы, генерирующий числовые значения независимо от таблиц. Используется для первичных ключей и других уникальных номеров. Основные параметры: START WITH, INCREMENT BY, MINVALUE, MAXVALUE, CYCLE/NO CYCLE, CACHE/NO CACHE. NEXTVAL возвращает следующее значение и увеличивает последовательность, CURRVAL возвращает последнее значение в текущем сеансе после первого NEXTVAL. Пропуски значений возможны при откатах и кэшировании.

11. Физическая структура СУБД. Пользовательский и серверный процессы. Экземпляр (Instance) СУБД, его структура.

Физическая работа Oracle строится на процессах и памяти. Пользовательский процесс запускает клиентское приложение и отправляет запросы. Серверный процесс обслуживает подключение, выполняет SQL и возвращает результат. Экземпляр Oracle - это SGA плюс фоновые процессы. SGA включает database buffer cache, shared pool, redo log buffer и другие области; фоновые процессы DBWn, LGWR, CKPT, SMON, PMON, ARCn обеспечивают запись, восстановление, контроль и архивирование.

12. Физическая структура базы данных.

Физическая структура базы данных - это набор файлов ОС, из которых состоит БД. Основные файлы: data files с данными таблиц и индексов, control files со структурой базы и состоянием файлов, online redo log files с журналом изменений. Дополнительно используются archived redo logs, файлы параметров PFILE/SPFILE, password file, trace и alert log. Потеря разных файлов требует разных процедур восстановления.

13. Создание и удаление экземпляра и базы данных.

Экземпляр и база создаются средствами DBCA или вручную: задаются параметры, создаются файлы, выполняется CREATE DATABASE, создаются словарь данных и служебные схемы. Экземпляр запускается STARTUP NOMOUNT для чтения параметров, затем база монтируется и открывается. Удаление выполняют через DBCA или вручную после остановки: удаляют службы, файлы данных, управляющие файлы, журналы, параметры и записи конфигурации. Перед удалением нужна резервная копия, если данные могут понадобиться.

14. Словарь данных. Предназначение и структура. Примеры представлений словаря данных.

Словарь данных - системная информация о структуре и состоянии базы: пользователи, таблицы, столбцы, права, ограничения, файлы, сегменты. Базовые таблицы словаря принадлежат SYS, пользователи обычно работают с представлениями. USER_* показывает объекты текущего пользователя, ALL_* - доступные объекты, DBA_* - всю базу при наличии прав. Примеры: USER_TABLES, USER_TAB_COLUMNS, DBA_USERS, DBA_TABLESPACES, DBA_DATA_FILES, USER_CONSTRAINTS.

15. Динамические представления производительности. Примеры. Доступность представлений на разных этапах загрузки экземпляра.

Динамические представления производительности V\$ показывают текущее состояние экземпляра и БД. Они основаны на памяти экземпляра и управляющих файлах. Примеры: V\$INSTANCE, V\$SESSION, V\$PROCESS, V\$DATABASE, V\$DATAFILE, V\$LOG, V\$PARAMETER, V\$TRANSACTION. На NOMOUNT доступны данные об экземпляре и параметрах, после MOUNT - сведения из control file о базе и файлах, после OPEN - полная информация о сессиях, объектах и активности.

16. Этапы выполнения запросов SELECT и команд DML. Чтение из базы данных, запись в базу данных.

SQL проходит parsing, optimization, row source generation и execution. При SELECT сервер ищет блоки в buffer cache; если их нет, читает с диска в память и возвращает строки. При DML также читаются нужные блоки, создаются undo-записи для отката и согласованного чтения, формируются

redo-записи для восстановления, затем блоки меняются в buffer cache. LGWR записывает redo, DBWn позже записывает измененные блоки данных на диск.

17. Этапы запуска экземпляра и открытия базы данных, этапы и способы остановки экземпляра и закрытия базы.

STARTUP проходит стадии NOMOUNT, MOUNT, OPEN. NOMOUNT запускает экземпляр по файлу параметров; MOUNT открывает управляющие файлы и связывает экземпляр с базой; OPEN открывает файлы данных и redo logs для работы пользователей. Остановка: SHUTDOWN NORMAL ждет отключения пользователей, TRANSACTIONAL ждет завершения транзакций, IMMEDIATE откатывает активные транзакции и закрывает базу, ABORT прерывает сразу и требует последующего instance recovery.

18. Файлы параметров. Способы изменения параметров экземпляра и вывода текущих значений параметров.

Параметры экземпляра хранятся в текстовом PFILE или серверном бинарном SPFILE. Текущие значения смотрят через SHOW PARAMETER, V\$PARAMETER, V\$SPPARAMETER.

Динамические параметры можно менять ALTER SYSTEM SET ... SCOPE=MEMORY, SPFILE или BOTH. Статические параметры требуют изменения файла параметров и перезапуска экземпляра. PFILE редактируется вручную, SPFILE изменяется командой ALTER SYSTEM.

19. Журнальные группы (REDO LOG) и элементы журнальных групп. Процесс записи в журнальные группы. Режимы ARCHIVELOG и NOARCHIVELOG.

Online redo log хранит записи обо всех изменениях для восстановления. Он состоит из групп, каждая группа может иметь несколько одинаковых членов для отказоустойчивости. LGWR пишет redo последовательно в текущую группу; при заполнении происходит log switch. В ARCHIVELOG заполненные группы архивируются ARCn и позволяют восстановление до точки сбоя. В NOARCHIVELOG группы перезаписываются без архивации, поэтому восстановление возможно только до последней полной копии.

20. Логическая и физическая структуры базы данных. Табличные пространства, сегменты, экстенды, блоки данных.

Логическая структура Oracle: табличные пространства, сегменты, экстенды и блоки данных. Табличное пространство объединяет логические объекты хранения и состоит из файлов данных. Сегмент принадлежит объекту, например таблице или индексу. Экстент - группа соседних блоков, выделенная сегменту. Блок данных - минимальная единица чтения и записи Oracle. Физическая структура ОС представлена data files, control files и redo log files.

21. Табличные пространства. Виды табличных пространств. Создание и удаление табличных пространств. Автономный режим. Режим «только чтение».

Табличное пространство - логическая область хранения сегментов. Основные виды: постоянные, временные, undo, системные SYSTEM/SYSAUX, пользовательские. Создается CREATE TABLESPACE с файлом данных, удаляется DROP TABLESPACE, при необходимости INCLUDING CONTENTS AND DATAFILES. OFFLINE делает табличное пространство недоступным для операций, ONLINE возвращает его. READ ONLY запрещает изменения и удобен для архивных данных; READ WRITE возвращает обычный режим.

22. Временные табличные пространства. Создание, удаление, использование.

Привязка временного табличного пространства к пользователю. Действия при повреждении временных табличных пространств.

Временное табличное пространство используется для сортировок, hash join, GROUP BY, создания индексов и глобальных временных операций, когда памяти не хватает. Создается CREATE TEMPORARY TABLESPACE ... TEMPFILE, назначается пользователю ALTER USER ... TEMPORARY TABLESPACE или задается как default temporary tablespace. Удаляется DROP TABLESPACE. При повреждении tempfile обычно не восстанавливают из backup: его удаляют и создают заново, так как постоянные пользовательские данные там не хранятся.

23. Сегменты отката. Ручное и автоматическое управление сегментами отката.

Сегменты отката, в современной Oracle undo-сегменты, хранят старые версии изменяемых данных. Они нужны для ROLLBACK, согласованного чтения и восстановления незавершенных транзакций. При ручном управлении администратор создает rollback segments и управляет их использованием. При автоматическом управлении задается UNDO tablespace, а Oracle сам создает и распределяет undo-сегменты. Автоматический режим проще и является стандартным.

24. Временные сегменты. Индексные сегменты.

Временные сегменты создаются во временных табличных пространствах для операций сортировки, группировки, hash-операций, построения индексов и временных таблиц; после завершения операции они освобождаются или переиспользуются. Индексные сегменты хранят данные индекса, чаще всего B-tree структуру: ключевые значения и ссылки на строки. Индекс ускоряет поиск и проверку уникальности, но требует места и замедляет DML, потому что индекс тоже нужно поддерживать.

25. Табличные сегменты. Виды табличных сегментов. Структура табличного сегмента.

Табличный сегмент - область хранения данных таблицы. Он состоит из экстентов, а экстененты - из блоков данных. В блоках находятся заголовок, служебная информация, каталог строк и сами строки. Виды табличных сегментов: обычные heap tables, индекс-организованные таблицы, кластерные, секционированные, вложенные и временные таблицы. Рост сегмента происходит выделением новых экстенентов в табличном пространстве.

26. Режимы ARCHIVELOG и NOARCHIVELOG. Различия.

В ARCHIVELOG заполненные online redo logs сохраняются как архивные журналы. Это позволяет выполнять горячие резервные копии и восстанавливать базу до момента сбоя или заданной точки времени, но требует места и администрирования архивов. В NOARCHIVELOG журналы после заполнения перезаписываются, база проще в обслуживании, но при сбое можно восстановиться только до последней согласованной резервной копии, обычно холодной.

27. Режим ARCHIVELOG: получение информации об архивировании и местоположении архивных журналов. Процесс перехода в ARCHIVELOG.

Информацию об архивировании получают командами ARCHIVE LOG LIST, SHOW PARAMETER log_archive, а также из V\$DATABASE, V\$ARCHIVED_LOG, V\$ARCHIVE_DEST. Переход в ARCHIVELOG выполняется при закрытой базе: SHUTDOWN IMMEDIATE, STARTUP MOUNT,

ALTER DATABASE ARCHIVELOG, ALTER DATABASE OPEN. Перед включением задают местоположение архивов LOG_ARCHIVE_DEST_n и контролируют свободное место, иначе база может остановиться при невозможности архивирования.

28. Поддержка национальных языков. Параметры поддержки на сервере и на клиенте.

NLS определяет язык сообщений, названия месяцев и дней, территорию, форматы дат, чисел, валют и сортировку. На уровне базы важны NLS_CHARACTERSET и NLS_NCHAR_CHARACTERSET, их смотрят в NLS_DATABASE_PARAMETERS. На уровне экземпляра - NLS_INSTANCE_PARAMETERS, на уровне сеанса - NLS_SESSION_PARAMETERS. Клиентские настройки задаются NLS_LANG и могут переопределять формат отображения. ALTER SESSION SET NLS_DATE_FORMAT, NLS_LANGUAGE, NLS_TERRITORY меняет параметры текущего сеанса.

29. Способы изменения настройки языка и территории на клиенте СУБД Oracle.

На клиенте Oracle язык и территория задаются переменной NLS_LANG в формате LANGUAGE_TERRITORY.CHARACTERSET, например RUSSIAN_RUSSIA.AL32UTF8. В Windows значение может задаваться в реестре Oracle Home или переменных среды, в Linux - через export NLS_LANG. Для конкретного подключения можно использовать ALTER SESSION SET NLS_LANGUAGE, NLS_TERRITORY, NLS_DATE_FORMAT. Клиентские настройки влияют на отображение дат, чисел, сообщений и преобразование символов.

30. Элементы сетевой инфраструктуры СУБД. Настройка сетевого взаимодействия клиента с сервером.

Сетевая инфраструктура Oracle включает Oracle Net, listener, сервисы БД, tnsnames.ora, listener.ora и sqlnet.ora. Listener принимает входящие подключения и направляет их к нужному сервису. Клиент использует connect string или сетевой псевдоним из tnsnames.ora с HOST, PORT и SERVICE_NAME/SID. sqlnet.ora задает методы именования, таймауты, аутентификацию и шифрование. Проверка выполняется tnsping и пробным подключением sqlplus user/password@service.

31. Понятие безопасности данных. Конфиденциальность, доступность и целостность данных. Угрозы информационной безопасности.

Безопасность данных - защищенность информации от несанкционированного доступа, изменения, уничтожения и нарушения работы. Конфиденциальность означает доступ только уполномоченным субъектам. Целостность - сохранение корректности и непротиворечивости данных. Доступность - возможность получить данные и сервисы в нужное время. Угрозы: НСД, утечка, подмена, удаление, ошибки пользователей, сбои оборудования, вредоносное ПО, SQL-инъекции, отказ в обслуживании.

32. Основные принципы обеспечения безопасности информации.

Ключевые принципы: законность, системность, комплексность, непрерывность защиты, своевременность мер, минимальные привилегии, разграничение доступа, персональная ответственность, контроль и аудит, резервирование, эшелонированная защита. Защита должна

охватывать пользователей, приложения, СУБД, ОС, сеть и физическую инфраструктуру. Меры должны соответствовать ценности данных и актуальным угрозам, а не существовать формально.

33. Нарушители информационной безопасности. Классификация нарушителей. Структура модели нарушителя АИС.

Нарушитель ИБ - лицо или процесс, способный нарушить правила доступа или повредить данные. Нарушителей классифицируют как внутренних и внешних, случайных и умышленных, с разным уровнем полномочий, знаний, мотивации и ресурсов. Модель нарушителя АИС описывает цели, доступные каналы, права, технические возможности, знания системы, предполагаемые действия и ограничения. Она нужна для выбора адекватных механизмов защиты.

34. Механизмы защиты информации в СУБД.

Защита в СУБД включает идентификацию и аутентификацию пользователей, управление учетными записями, системные и объектные привилегии, роли, ограничения целостности, представления для ограничения видимости, аудит, шифрование данных и сетевого трафика, резервное копирование и восстановление. Дополнительно применяются политики паролей, профили ресурсов, блокировка учетных записей, мандатное разграничение через Label Security и контроль действий администраторов.

35. Модели управления доступом. Особенности реализации мандатной модели доступа в СУБД.

Основные модели доступа: дискреционная DAC, где владелец объекта выдает права; мандатная MAC, где доступ определяется метками безопасности; ролевая RBAC, где права назначаются ролям. В СУБД мандатная модель реализуется через метки субъектов и объектов, уровни допуска, категории и правила сравнения меток. Пользователь не может произвольно передавать права, а строки с более высокой меткой скрываются или запрещаются для изменения.

36. Проблемы многоэкземпляльности и проблемы использования индексов при реализации мандатной модели доступа в СУБД.

Многоэкземпляльность возникает, когда один логический объект имеет несколько версий с разными метками безопасности: пользователи разных уровней видят разные данные. Это усложняет обеспечение уникальности, обновление и интерпретацию ключей. Индексы в мандатной модели должны учитывать метку строки, иначе через наличие или отсутствие индексной записи можно раскрыть скрытые данные. Поэтому индексы строятся с учетом меток, что увеличивает объем, сложность поиска и сопровождение.

37. Создание пользователей. Способы аутентификации пользователей и администраторов.

Пользователь создается CREATE USER с указанием пароля, табличных пространств, квот, профиля и состояния ACCOUNT LOCK/UNLOCK. Обычная аутентификация - парольная в СУБД. Внешняя аутентификация доверяет ОС, глобальная использует каталог/службу идентификации. Администраторы SYSDBA/SYSOPER могут аутентифицироваться через ОС-группы или password file. После создания пользователю обычно выдают CREATE SESSION и нужные привилегии.

38. Управление пользователями. Системные привилегии. Предоставление и отмена системных привилегий. Выдача прав с правом наследования.

Системные привилегии разрешают действия в БД в целом: CREATE SESSION, CREATE TABLE, CREATE USER, ALTER SYSTEM, CREATE ANY INDEX и т.п. Они выдаются GRANT privilege TO user_or_role и отменяются REVOKE privilege FROM user_or_role. WITH ADMIN OPTION позволяет получателю передавать системную привилегию или роль другим и отменять ее у них. Управление пользователями также включает ALTER USER, блокировку, смену пароля, квоты и профили.

39. Управление пользователями. Объектные привилегии. Предоставление и отмена объектных привилегий. Выдача прав с правом наследования.

Объектные привилегии дают доступ к конкретному объекту схемы: SELECT, INSERT, UPDATE, DELETE для таблиц и представлений, EXECUTE для процедур, ALTER/INDEX/REFERENCES для таблиц. Выдаются GRANT privilege ON object TO user_or_role, отменяются REVOKE privilege ON object FROM user_or_role. WITH GRANT OPTION позволяет получателю передавать объектную привилегию другим. Для UPDATE и SELECT можно ограничивать доступ отдельными столбцами.

40. Методы аутентификации в СУБД. В каком виде и где хранятся пароли для пользователей и администраторов.

Методы аутентификации Oracle: парольная в БД, внешняя через ОС, глобальная через каталог, а также административная через ОС или password file. Пароли обычных пользователей не хранятся открыто: в словаре данных находятся хэши/верификаторы паролей. Административные пользователи SYSDBA/SYSOPER могут проверяться по password file, где также хранятся защищенные хэшированные сведения. При внешней аутентификации пароль хранится и проверяется вне СУБД.

41. Роли. Создание, предоставление и отмена привилегий через роли. Роли с паролем.

Роль - именованный набор привилегий, упрощающий управление доступом. Создается CREATE ROLE, привилегии выдаются роли через GRANT, роль выдается пользователю или другой роли, отменяется REVOKE. Пользователь получает права активных ролей в сеансе; роли можно включать SET ROLE. Роль с паролем создается CREATE ROLE role IDENTIFIED BY password и активируется SET ROLE role IDENTIFIED BY password. Роли удобны для групп пользователей с одинаковыми обязанностями.

42. Система протоколирования (аудит). Включение/отключение аудита. Определение событий аудита. Просмотр результатов протоколирования.

Аудит фиксирует действия пользователей и администраторов: входы, DDL, DML, обращения к объектам, использование привилегий. Включается параметром AUDIT_TRAIL или политиками Unified Auditing, события задаются AUDIT, отключаются NOAUDIT. Аудит может быть по действию, объекту, привилегии, успешным или неуспешным попыткам. Результаты смотрят в DBA_AUDIT_TRAIL, USER_AUDIT_TRAIL, DBA_OBJ_AUDIT_OPTS или UNIFIED_AUDIT_TRAIL.

43. Ограничение целостности. Способы создания и отмены ограничений целостности. Включение и отключение ограничений целостности.

Ограничения целостности поддерживают корректность данных: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK. Их создают внутри CREATE TABLE или добавляют ALTER TABLE

ADD CONSTRAINT. Удаляют ALTER TABLE DROP CONSTRAINT, для NOT NULL - ALTER TABLE MODIFY column NULL. Ограничение можно отключить DISABLE и включить ENABLE; при включении Oracle проверяет существующие данные, если используется VALIDATE. NOVALIDATE оставляет старые строки без проверки, но контролирует новые.

44. Ограничения целостности Primary key и Unique key. Их связь с индексами.

PRIMARY KEY задает главный идентификатор строки: значения должны быть уникальными и не NULL. UNIQUE KEY требует уникальности значений, но допускает NULL в зависимости от правил СУБД. Для обеспечения PRIMARY KEY и UNIQUE Oracle создает или использует уникальный индекс. Если подходящий индекс уже есть, он может быть использован ограничением. При отключении или удалении ограничения поведение связанного индекса зависит от способа создания и параметров.

45. Ограничение целостности Foreign key.

FOREIGN KEY обеспечивает ссылочную целостность между дочерней и родительской таблицами. Значение внешнего ключа должно соответствовать существующему PRIMARY KEY или UNIQUE KEY родительской таблицы либо быть NULL. При попытке удалить родительскую строку с дочерними ссылками возникает ошибка, если не задано ON DELETE CASCADE или ON DELETE SET NULL. Индекс на внешнем ключе не обязателен, но часто нужен для производительности и уменьшения блокировок.

46. Понятие транзакции. Команды управления транзакциями. Явная и неявная обработка транзакций.

Транзакция - логическая единица работы с БД, которая должна выполняться целиком или отменяться целиком. Свойства ACID: атомарность, согласованность, изолированность, долговечность. Команды управления: COMMIT фиксирует изменения, ROLLBACK отменяет, SAVEPOINT задает точку частичного отката, SET TRANSACTION задает режим. Явная обработка - пользователь сам выполняет COMMIT/ROLLBACK. Неявная фиксация происходит при DDL, завершении сеанса без ошибок или некоторых административных действиях.

47. Транзакции. Физическая реализация механизма транзакций в СУБД Oracle.

Oracle реализует транзакции через undo, redo, блокировки и SCN. Undo хранит старые версии строк для отката и согласованного чтения. Redo фиксирует описание изменений для восстановления после сбоя. SCN задает логический момент времени и помогает строить согласованное чтение. Блокировки защищают изменяемые строки от конфликтующих изменений. При COMMIT записывается redo о фиксации, транзакция становится долговечной; при ROLLBACK изменения восстанавливаются по undo.

48. Транзакции. Блокировки сеансов в процессе транзакции. Транзакция read only.

При DML Oracle ставит строковые блокировки на изменяемые строки и табличные блокировки совместимого типа. Другие сеансы могут читать старую согласованную версию, но конфликтующее изменение будет ждать завершения первой транзакции. Долгие транзакции увеличивают удержание undo и риск блокировок. Транзакция READ ONLY задается SET TRANSACTION READ ONLY: она видит согласованный снимок данных на момент начала и запрещает DML в текущем сеансе.

49. Шифрование на уровне баз данных и на уровне сети.

На уровне базы применяют Transparent Data Encryption для шифрования табличных пространств или отдельных столбцов; ключи хранятся в wallet/keystore. Это защищает данные на диске, backup и файлы при краже носителя. На уровне сети шифруют трафик между клиентом и сервером средствами Oracle Net Native Network Encryption или TLS/SSL. Сетевое шифрование защищает от перехвата, но не заменяет разграничение доступа и аудит.

50. Механизмы Label Security для реализации мандатной модели разграничения доступа.

Oracle Label Security реализует мандатное разграничение на уровне строк. Создается политика безопасности, уровни, категории и группы, пользователям назначаются метки допуска, а строкам - метки данных. При SELECT, INSERT, UPDATE, DELETE политика автоматически сравнивает метку пользователя и метку строки. Пользователь видит и изменяет только данные, разрешенные его уровнем и категориями. Это позволяет совмещать MAC с обычными привилегиями Oracle.

51. SQL-инъекции. Определение, типы. Методы противодействия.

SQL-инъекция - внедрение злоумышленником SQL-кода во входные данные приложения, если они напрямую подставляются в запрос. Типы: классическая, UNION-based, error-based, blind boolean-based, time-based, stacked queries. Последствия: обход авторизации, чтение и изменение данных, удаление объектов. Защита: параметризованные запросы и bind variables, запрет конкатенации SQL, проверка ввода, минимальные права учетной записи, безопасные ошибки, хранимые процедуры с параметрами, обновления и WAF.

52. Методы резервирования баз данных в режиме ARCHIVELOG. Преимущества и недостатки.

В ARCHIVELOG доступны холодные и горячие backup, RMAN backup, инкрементные копии и копирование отдельных файлов/табличных пространств. Главное преимущество - восстановление до момента сбоя или заданной точки времени при наличии backup и архивных журналов; база может оставаться открытой при горячем резервировании. Недостатки: нужны место под архивы, контроль их сохранности, настройка RMAN/политик хранения и дополнительная нагрузка.

53. Методы резервирования баз данных в режиме NOARCHIVELOG. Преимущества и недостатки.

В NOARCHIVELOG надежный backup обычно выполняется при закрытой базе: копируются data files, control files и redo logs или используется RMAN при MOUNT. Преимущества: простота, меньше места, не нужно хранить архивные журналы. Недостатки: база недоступна во время согласованной копии, невозможно восстановление до момента сбоя, все изменения после последнего backup теряются. Режим подходит для учебных или некритичных баз.

54. Порядок полного восстановления открытой базы данных в режиме ARCHIVELOG после сбоя диска.

Сначала определить поврежденный файл по ошибкам, alert log и V\$DATAFILE/V\$RECOVER_FILE. Если база открыта, перевести поврежденный datafile или tablespace OFFLINE. Затем восстановить файл из резервной копии RESTORE DATAFILE/TABLESPACE и применить архивные и текущие redo RECOVER

DATAFILE/TABLESPACE. После успешного восстановления вернуть файл ONLINE. Для системных файлов или SYSTEM tablespace часто требуется MOUNT и восстановление при закрытой базе.

55. Особенности восстановления базы данных при потере табличного пространства, если экземпляр находится в режиме STARTUP и SHUTDOWN.

Если база остановлена и обнаружена потеря файла табличного пространства, в ARCHIVELOG обычно запускают экземпляр в MOUNT, восстанавливают поврежденный datafile из backup, выполняют RECOVER DATAFILE или RECOVER TABLESPACE и открывают базу. В NOARCHIVELOG восстановление обычно возможно только возвратом всей базы к последней согласованной холодной копии. Если база уже OPEN, пользовательское табличное пространство можно перевести OFFLINE, восстановить и вернуть ONLINE без остановки всей БД.

56. Особенности восстановления базы данных при потере табличного пространства, для которого отсутствует BACKUP.

Если backup табличного пространства отсутствует, восстановление зависит от его типа. Для временного tablespace достаточно пересоздать tempfile. Для пользовательского tablespace без backup полное восстановление данных обычно невозможно; можно удалить и создать tablespace заново или восстанавливать данные логическим импортом, если он есть. Для SYSTEM, SYSAUX или UNDO потеря без backup критична и может потребовать полного пересоздания базы. ARCHIVELOG не заменяет исходную резервную копию файла.

57. Порядок и виды неполного восстановления базы данных.

Неполное восстановление возвращает базу не к последнему состоянию, а к заданной точке. Виды: UNTIL TIME, UNTIL SCN, UNTIL SEQUENCE, восстановление с резервной копией control file. Общий порядок: остановить БД, восстановить нужные файлы из backup, открыть MOUNT, выполнить RECOVER DATABASE UNTIL ..., затем открыть ALTER DATABASE OPEN RESETLOGS. После RESETLOGS начинается новая инкарнация базы, поэтому нужен новый полный backup.

58. Утилита Recovery Manager. Структура, основные возможности. Достоинства и недостатки.

RMAN - встроенная утилита Oracle для backup, restore и recover. Структура: клиент RMAN, целевая база, каналы, репозиторий в control file и необязательный recovery catalog. Возможности: полные и инкрементные backup, восстановление базы, tablespace, datafile и блоков, проверка backup, сжатие, шифрование, удаление устаревших копий, применение архивных журналов. Плюсы: автоматизация и меньше ошибок. Минусы: требует настройки, места, контроля политик хранения и навыков восстановления.

59. Логическое резервирование с помощью утилит экспорта/импорта.

Логическое резервирование сохраняет объекты и данные на уровне SQL, а не физические файлы БД. Используются Export/Import или Data Pump expdp/impdp, результат обычно dump-файл. Режимы: экспорт таблиц, схем, tablespace или всей базы. Плюсы: переносимость между базами и платформами, удобство восстановления отдельных объектов, работа при открытой базе. Минусы:

не заменяет физический backup, медленнее для больших БД, не всегда сохраняет все физические свойства и не подходит для полного восстановления после серьезного сбоя.

60. Место средств защиты СУБД в составе подсистемы защиты АИС. Взаимосвязь механизмов защиты СУБД и аналогичных механизмов других компонентов АИС.

Защита СУБД - часть общей защиты АИС наряду с защитой сети, ОС, приложений и физической инфраструктуры. Сеть ограничивает доступ к портам и шифрует каналы, ОС защищает файлы БД и процессы, приложение контролирует бизнес-логику, СУБД обеспечивает точные права на данные, аудит, целостность и восстановление. Механизмы должны дополнять друг друга: единая аутентификация, согласованные роли, SIEM для журналов, TLS/VPN для трафика, TDE и шифрование дисков для хранения, резервное копирование на уровне БД и инфраструктуры.